

EE8257 Information Security



Lecture 4 Symmetric Key Cryptography

P. S. Ranaweera

E313

Department of Electrical and Information Engineering

Faculty of Engineering

University of Ruhuna



Outline

- Symmetric Key Cryptography
- Stream Ciphers
 - A5/1
 - RC4
- Block Ciphers
 - DES
 - T-DES
 - AES
 - TEA
- Block Cipher Modes
- Integrity protection with Symmetric key crypto

Symmetric Key Crypto

- Same key is used for encryption and decryption
- Key should be shared between the parties engaged in communication
- Key should be changed from time to time to prevent its exposure to unwanted parties
- Two types of symmetric key ciphers :
 - Stream ciphers
 - Block ciphers

Stream Ciphers

- Encryption is done for one digit / bit at a time
- A relatively shorter key is used (key K of n bits)
- Key is stretched into a larger key stream

$$\text{StreamCipher}(K) = S ; m \text{ bits } (m > n)$$

- **XOR** operations are carried out to obtain the cipher text from plaintext and vice versa... (same as OTP)

- If $P = p_1 p_2 p_3 \dots p_m$, $S = s_1 s_2 s_3 \dots s_m$
- Encryption: $c_0 = p_0 \oplus s_0, c_1 = p_1 \oplus s_1, c_2 = p_2 \oplus s_2, \dots, c_m = p_m \oplus s_m$
- Decryption: $p_0 = c_0 \oplus s_0, p_1 = c_1 \oplus s_1, p_2 = c_2 \oplus s_2, \dots, p_m = c_m \oplus s_m$

Stream Ciphers (Cont...)

- Types of stream ciphers :
 - A5/1
 - A5/2
 - A5/3 (KASUMI)
 - RC₄

A5/1

software oriented



- Employs 3 Linear Feedback Shift Registers (LFSR)

- X register : 19 bits $\rightarrow x_0, x_1, x_2, \dots, x_{18}$
 - Y register : 22 bits $\rightarrow y_0, y_1, y_2, \dots, y_{21}$
 - Z register : 23 bits $\rightarrow z_0, z_1, z_2, \dots, z_{22}$
- } $19 + 22 + 23 = 64$ bits

- Key is 64 bits

- Key is used as the initial filled bit stream of the registers

Internal memory units that store and update bits.

- Keystream is generated from the filled registers

Bit stream generated from the registers, used for encryption/decryption

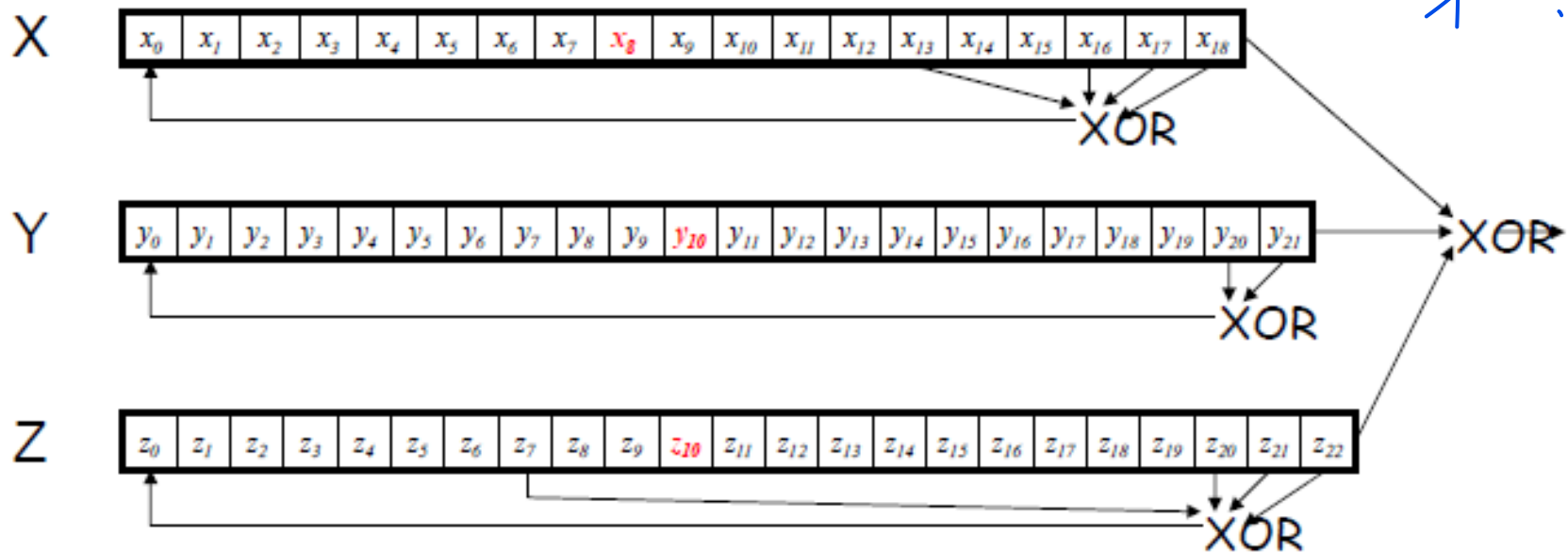
A5/1

maj = major value(highest occurrence)

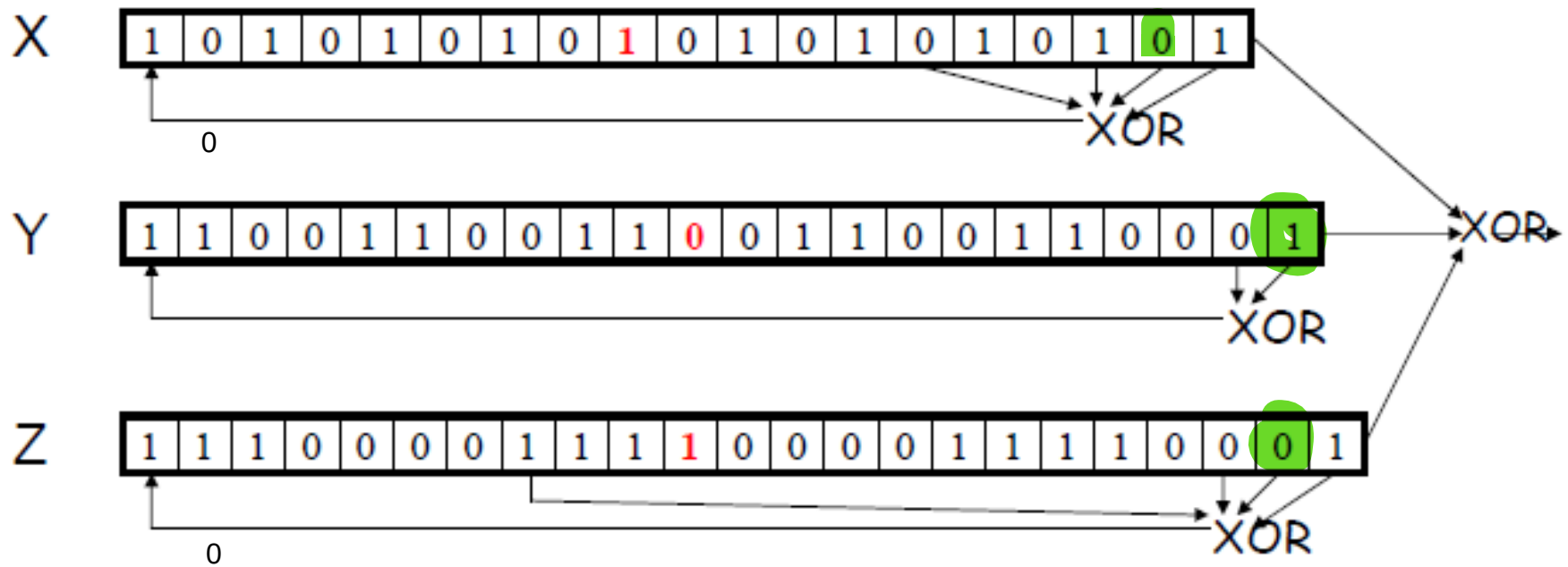
- At each step: $m = maj(x_8, y_{10}, z_{10})$
 - Examples: $maj(0,1,0) = 0$ and $maj(1,0,1) = 1$
- If $x_8 = m$ then **X** steps
$$t = x_{13} \oplus x_{16} \oplus x_{17} \oplus x_{18}$$
$$x_i = x_i - 1 \text{ for } i = 18, 17, \dots, 1 \text{ and } x_0 = t$$
- If $y_{10} = m$ then **Y** steps
$$t = y_{20} \oplus y_{21}$$
$$y_i = y_i - 1 \text{ for } i = 21, 20, \dots, 1 \text{ and } y_0 = t$$
- If $z_{10} = m$ then **Z** steps
$$t = z_7 \oplus y_{20} \oplus y_{21} \oplus y_{22}$$
$$z_i = z_i - 1 \text{ for } i = 22, 21, \dots, 1 \text{ and } z_0 = t$$
- Keystream bit is $\rightarrow x_{18} \oplus y_{21} \oplus z_{22}$

A5/1

- Each value is a single bit
- Key is used as **initial fill** of registers
- Each register steps or not, based on (x_8, y_{10}, z_{10})
- Keystream bit is XOR of right bits of registers



A5/1



- In this example, $m = \text{maj}(x_8, y_{10}, z_{10}) = \text{maj}(1, 0, 1) = 1$
- Register **X** steps, **Y** does not step, and **Z** steps
- Keystream bit is XOR of right bits of 3 registers
- Here, keystream bit will be $\rightarrow 0 \oplus 1 \oplus 0 = 1$ XOR of final shifted result (end bit)

A5/1

- Stream ciphers are implemented using hardware
- The process to generate a single keystream bit seems to be complicated
- Due to hardware implementation → bit generation matches with the clock speed
- In the past, they were popular
- Outran by the software based crypto systems due to faster processors in recent times
- Still some applications uses these (E.g.: GSM)

RC4

A5/1 a bit at a time but RC4 a byte at a time

- Algorithm is optimized for software implementation (unlike A5/1) 
- Entire algorithm is **byte** based
- Produces a keystream byte at each step (A5/1 $\rightarrow$ 1 bit)
- Employs a self-modifying **lookup table**
- Table always contains some permutation of $0, 1, \dots, 255$
- Initialize the permutation using the key $\rightarrow key[i] ; i = 0, 1, 2, \dots, n-1$
 - Each $key[i]$ is a byte
- At each step, RC4
 - Swaps elements in current lookup table
 - Selects a keystream byte from table

RC4 (Cont...)

not going in detail in RC4 just a overall idea

→ hardware oriented

- $s[i]$ is a permutation 0,1,...,255
- Key can be of any length from 0 to 256
- Key is used to initialize the lookup table or
- Algorithm initiation in pseudo-code : $s[]$



```
for i = 0 to 255
    S[i] = i
    K[i] = key[i mod N]
next i
j = 0
for i = 0 to 255
    j = (j + S[i] + K[i]) mod 256
    swap(S[i], S[j])
next i
i = j = 0
```

swapping is done
K is actually driving j

RC4 (Cont...)

- Keystream byte generation:

```
i = (i + 1) mod 256
j = (j + S[i]) mod 256
swap(S[i], S[j])
t = (S[i] + S[j]) mod 256
keystreamByte = S[t]
```

- Keystream bytes is used as a OTP
- First 256 keystream bytes are discarded 
- RC4 is applications:
 - Pseudo random number generators
 - SSL
- “Death of stream ciphers”

RSA- Rivest/shammy...

Block Ciphers

- Based on codebook concept
- Codebook is determined by the key
- Employ both confusion and diffusion concepts
- Plaintext and ciphertext are divided into fixed size blocks (padding might be employed)
- Encryption is done one block after another
- Encryption is carried out by an *Iterative Round Function*

Block Ciphers (Cont...)

- Output of a certain stage is related to the input of the next stage
- Modern day block ciphers are based on *Feistel cipher* designs
- Block cipher design is based on :
 - Expansion
 - Permutation
 - Substitution
 - XOR

Block Ciphers (Cont...)

- Types of block ciphers :
 - Data Encryption Standard (DES)
 - Triple DES – has 2 keys
 - Advanced Encryption Standard (AES)
 - RC₅
 - International Data Encryption Algorithm (IDEA)
 - Blowfish

Feistel Cipher

if the round function is zero we get plain text itself or swapped plaintext

- Feistel cipher is a block cipher design, not a cipher
- Named after *Horst Feistel*
- Plaintext block is split into left and right block halves

$$P = (L_0, R_0)$$

Encryption:

- For each round $i = 1, 2, \dots, n$

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

F is the round function

K_i is the subkey for round i

- Ciphertext $\rightarrow C = (L_n, R_n)$

Round Function

Feistel Cipher

Decryption :

- Ciphertext $\rightarrow C = (L_n, R_n)$
- For each round $i = n, n-1, \dots, 1$

$$R_{i-1} = L_i$$

$$L_{i-1} = R_i \oplus F(R_{i-1}, K_i)$$

- Plaintext $\rightarrow P = (L_0, R_0)$
- Any round function would be applicable for feistel ciphers
- But, all the functions would not be secure

Data Encryption Standard(DES)

- Originated in 1970s  another word for devil
- Based on Lucifer cipher (feistel cipher) developed by IBM
- US government involved NSA (National Security Agency) in modifying the Lucifer cipher into DES
- Changes done to Lucifier cipher:
 - Key length changed from 128 to 64 (56 with discarding 8 bits) ➔ Easier to break with brute force attack
 - Substitution boxes (S-box) ➔ strengthen the algorithm

DES

L13 and R16 are cipher text

- DES is a feistel cipher with 16 rounds
- 64bit block size
- 56 bit key size
- 48 bit sub-key for each round
- 8 S-boxes
- Each S-box maps 6bits to 4bits (48 bits → 32 bits)
- Permutation Boxes (P-box)

DES (Cont...)

$$P_i = (L_i, R_i)$$

- **Round Function :**

$$F(R_{i-1}, K_i) = P\text{-box}(S\text{-boxes}(\text{Expand}(R_{i-1} \oplus K_i)))$$

- **Key Schedule:**

$$K_i = \text{Compress}(\text{Shift}(\text{Segment}(\text{key}_{i-1})))$$

$$\text{key}_i = \text{Shift}(\text{Segment}(\text{key}_{i-1}))$$

- **Outcome of a round :**

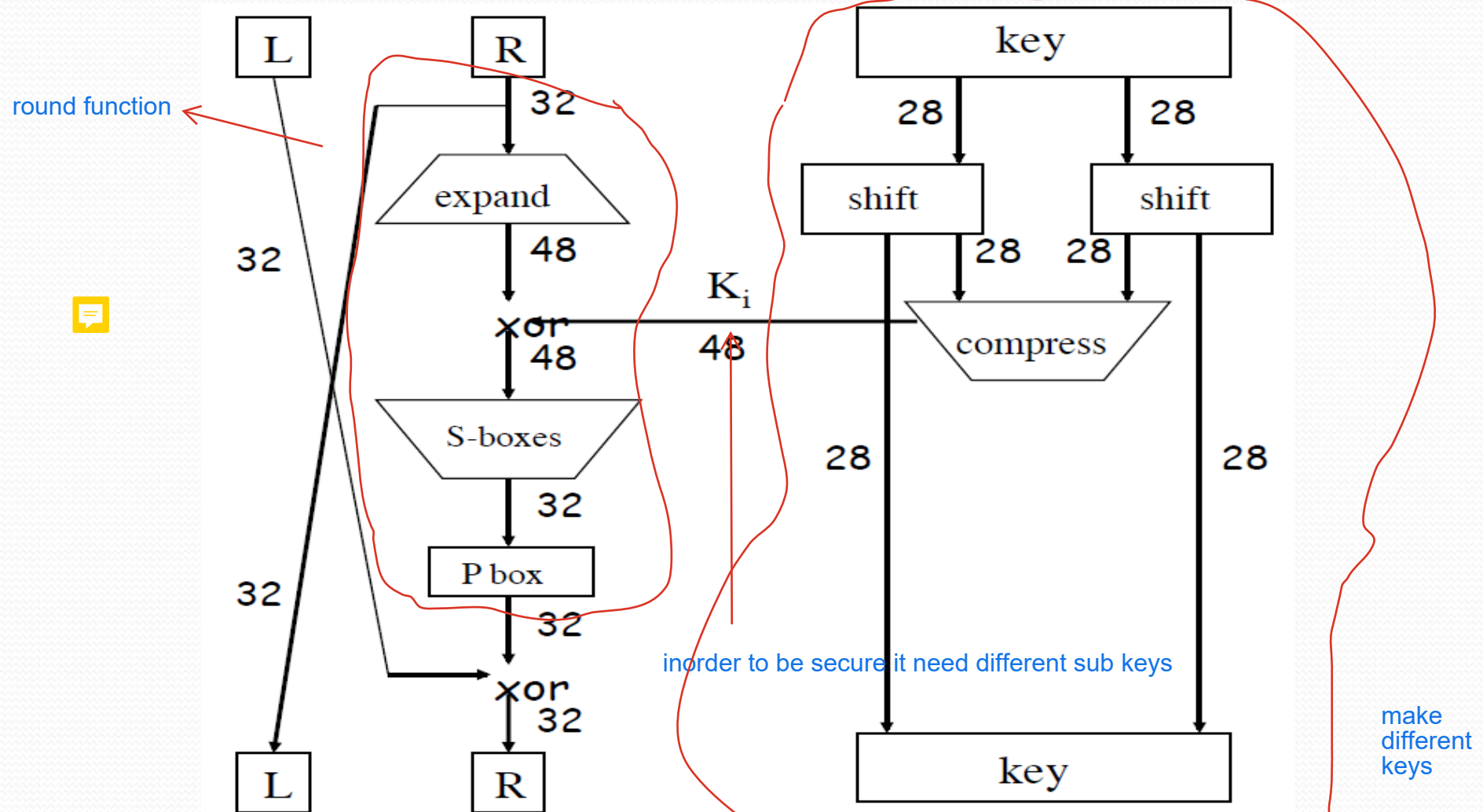
$$R_i = F(R_{i-1}, K_i) \oplus L_{i-1}$$

$$L_i = R_{i-1}$$

DES Round

this is 1 round there are 16 like this

as R is 32 and key size is 48 we cant do XOR due to size difference



DES Expansion

- Input 32 bits :

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

- Output 48 bits :

31	0	1	2	3	4	3	4	5	6	7	8
7	8	9	10	11	12	11	12	13	14	15	16
15	16	17	18	19	20	19	20	21	22	23	24
23	24	25	26	27	28	27	28	29	30	31	0

DES S-box

- 8 S – boxes
- Each maps 6 bits to 4 bits
- If the input bits are numbered from 0 to 5 (0,1,2,3,4,5)
- S-Box 1

$b_1b_2b_3b_4$

		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
b_0b_5	00	1110	0100	1101	0001	0010	1111	1011	1000	0011	1010	0110	1100	0101	1001	0000	0111
	01	0000	1111	0111	0100	1110	0010	1101	0001	1010	0110	1100	1011	1001	0101	0011	1000
	10	0100	0001	1110	1000	1101	0110	0010	1011	1111	1100	1001	0111	0011	1010	0101	0000
	11	1111	1100	1000	0010	0100	1001	0001	0111	0101	1011	0011	1110	1010	0000	0110	1101

if 1 0 1 1 0 1
b0 b1 b2 b3 b4 b5

output

DES P-box

does not add anything to make output just change the order

- Input 32 Bits:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

- Output 32 Bits:

15	6	19	20	28	11	27	16	0	14	22	25	4	17	30	9
1	7	23	13	31	26	2	8	18	12	29	5	21	10	3	24

DES Sub key Generation Algorithm

Sub Key Generation Algorithm :

- Segmenting
 - Permuting
 - Permuting with r_i
 - Compression and Permuting
- } Shifting

DES Sub key Segmenting and Permuting

- DES key - 56 bits
- Permuted Left half key 28 bits → LK

49	42	35	28	21	14	7
0	50	43	36	29	22	15
8	1	51	44	37	30	23
16	9	2	52	45	38	31

- Permuted Right half key 28 bits → RK

55	48	41	34	27	20	13
6	54	47	40	33	26	19
12	5	53	46	39	32	25
18	11	4	24	17	10	3

DES Sub key Compression

skipped

- Left half permutation $\rightarrow LP$

13	16	10	23	0	4	2	27	14	5	20	9
22	18	11	3	25	7	15	6	26	19	12	1

- Right half permutation $\rightarrow RP$

12	23	2	8	18	26	1	11	22	16	4	19
15	20	10	27	5	24	17	13	21	7	0	3

DES Sub key Generation Algorithm

$$r_i = \begin{cases} 1, & \text{if } i \in \{1,2,9,16\} \\ 2 & \text{otherwise} \end{cases}$$

For each round $i = 1, 2, \dots, n$

LK = cyclically left shift LK by r_i bits

RK = cyclically left shift RK by r_i bits

The left half of subkey K_i consists of bits LP of LK

The right half of subkey K_i consists of bits RP of RK

next i

Other features of DES

- Initial permutation is applied to the plaintext before round one
- Inverse of the initial permutation is applied after the final round
- The halves of the resulting blocks would be swapped
- Ciphertext $\rightarrow (R_{16}, L_{16})$ left and right are swapped
- Does not serve any security purpose

xOR doesnt add security

Security in DES

- Any Linear component in a security algorithm can be effectively broken due to the mathematical involvement
- S-boxes are considered as the crucial aspect of security in DES due to its non-linearity
- Expansion permutation is also considered as security critical
- Other methods does not serve an effective security purpose (linear)
- DES is vulnerable → key is too small
- Shortcut attacks are possible

Block Cipher Notation

- Plaintext Block $\rightarrow P$
- Ciphertext Block $\rightarrow C$
- Key $\rightarrow K$
- Encryption :

$$C = E(P, K)$$

To denote encrypting we use E

- Decryption :

$$P = D(C, K)$$

same key is used for both

$$P = D(\underbrace{E(P, K)}_C, K)$$

C

$$\cancel{P} = E(\underbrace{D(C, K)}_P, K)$$

P

Double DES

becz of this issue this never went through

- Using a key pair K_1, K_2
- Double DES : $C = E(E(P, K_1), K_2)$
- Key pair collectively provides a 112 bit key
- There is a vulnerability → Chosen plaintext attack
 - Pre compute $E(P, K)$ for all possible $K \rightarrow 2^{56}$
 - Since we know C , we decrypt with $\tilde{K}$ until we find $E(P, K)$
 - If a match is found → $D(C, \tilde{K}) = E(P, K)$

stored in a dictionary the keys ?????

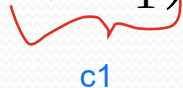
$$C = E(E(P, K), \tilde{K})$$

the attacker is going to decrypt with different keys until he get an match from the dictionary

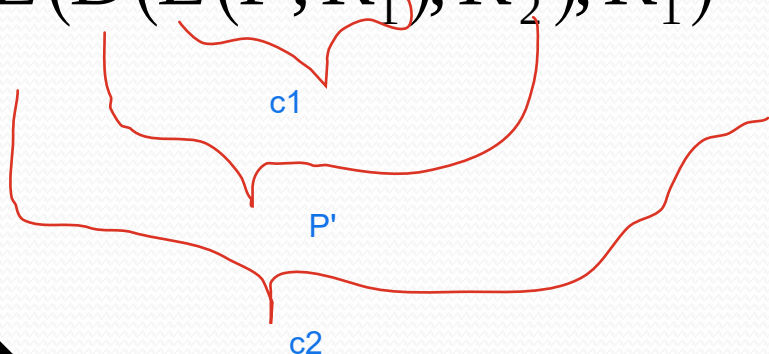
of the attacker knows one part of the key he can crack the rest

Triple DES (3DES)

- Logical approach :

$$C = E(E(E(P, K_1), K_2), K_3)$$


- Actual approach :

$$C = E(D(E(P, K_1), K_2), K_1)$$


- A key pair → 112 bit key
- EDE approach
- Backward compatibility →

if we use same key instead of different keys,
this is same as single DES

$$C = E(D(E(P, K), K), K) = E(P, K)$$

Advanced Encryption Standard (AES)

- Originated due to the limitations of DES → 56 bit key size was subjected to exhaustive key search
- NIST conducted a competition in 1990's
- NSA was involved and the process was open
- ***Rijndael (Rain Doll) Algorithm*** was selected

Advanced Encryption Standard (AES) (Cont...)

- AES is an iterated block cipher. But not a Feistel cipher.
- Block sizes : 128 b, 192 b, 256 b
- Key length : 128 b, 192 b, 256 b
- Key length and block size are independent
- Number of rounds → 10 to 14 (key length dependent)

AES Round

- Each round of AES performs 4 distinct functions which are categorized under three layers
 - ByteSub (non-linear layer)
 - ShiftRow (linear mixing layer)
 - MixColumn (non-linear layer)
 - AddRoundKey (Key addition layer)

AES ByteSub

- For 192 b block size → data is contained in a 4 x 6 byte array (4x6x8 b = 192 b)

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} \end{bmatrix}$$

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} \end{bmatrix} \xrightarrow{b_{ij} = \text{ByteSub}(a_{ij})} \begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} & b_{04} & b_{05} \\ b_{10} & b_{11} & b_{12} & b_{13} & b_{14} & b_{15} \\ b_{20} & b_{21} & b_{22} & b_{23} & b_{24} & b_{25} \\ b_{30} & b_{31} & b_{32} & b_{33} & b_{34} & b_{35} \end{bmatrix}$$

AES ByteSub (Cont...)

- ByteSub is acting as the AES S-box
- ByteSub is non-linear and invertible
- In simple terms → a lookup table (Hexadecimal)

$$\text{ByteSub}(6b) = 7f$$

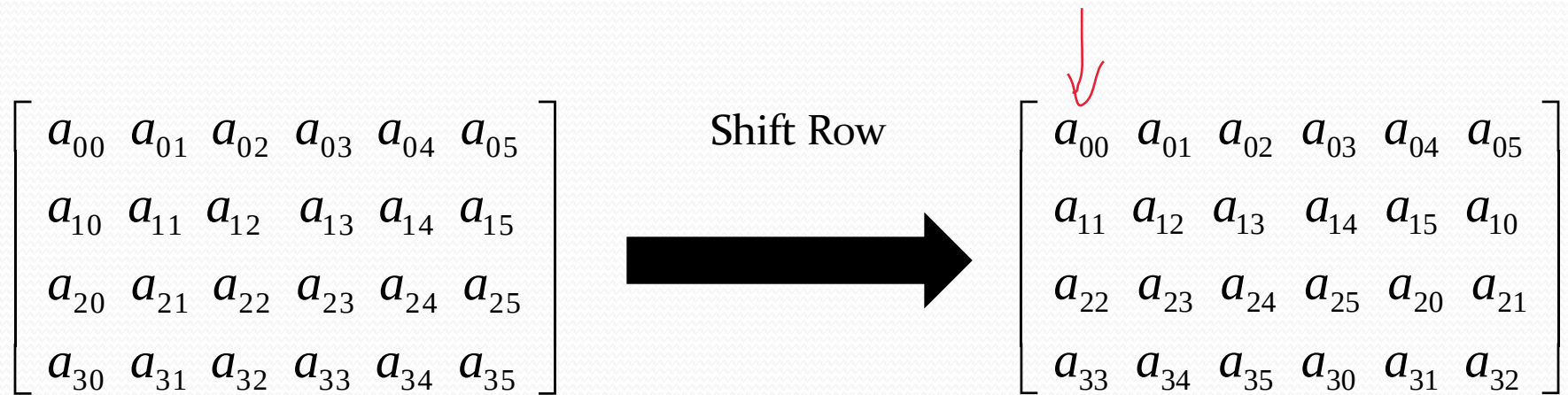
First 4
bits of
input

Last 4 bits of input

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

AES Shift Row

- ShiftRow operation is a simple cyclic shift of bytes in each row of the byte array

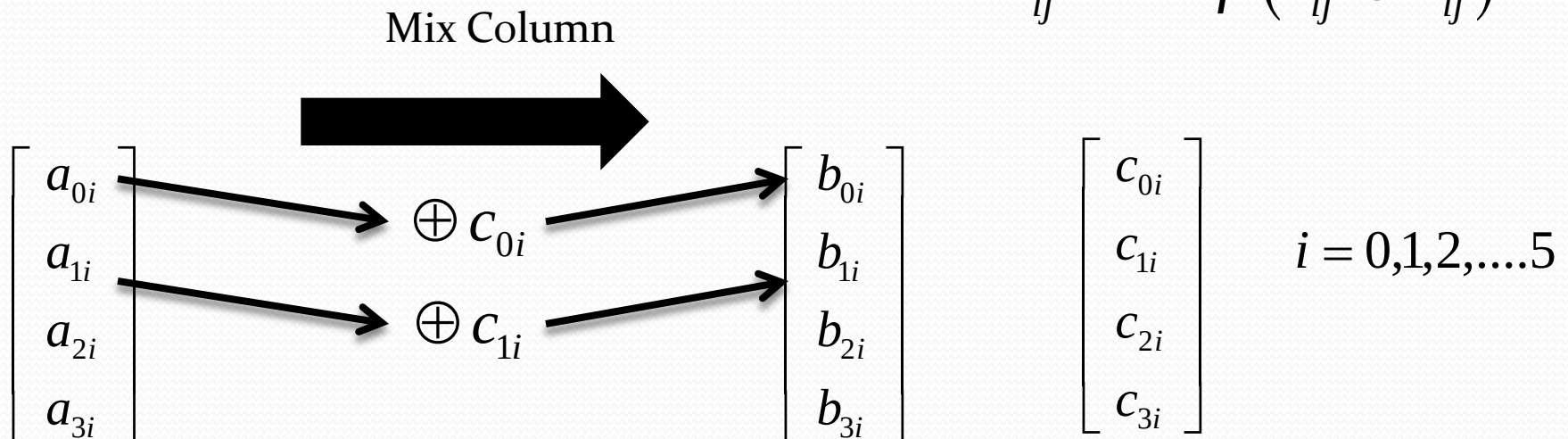


first row - not shifted
second row - shifted by 1
third row - shifted by 2
fourth row - shifted by 3

AES Mix Column

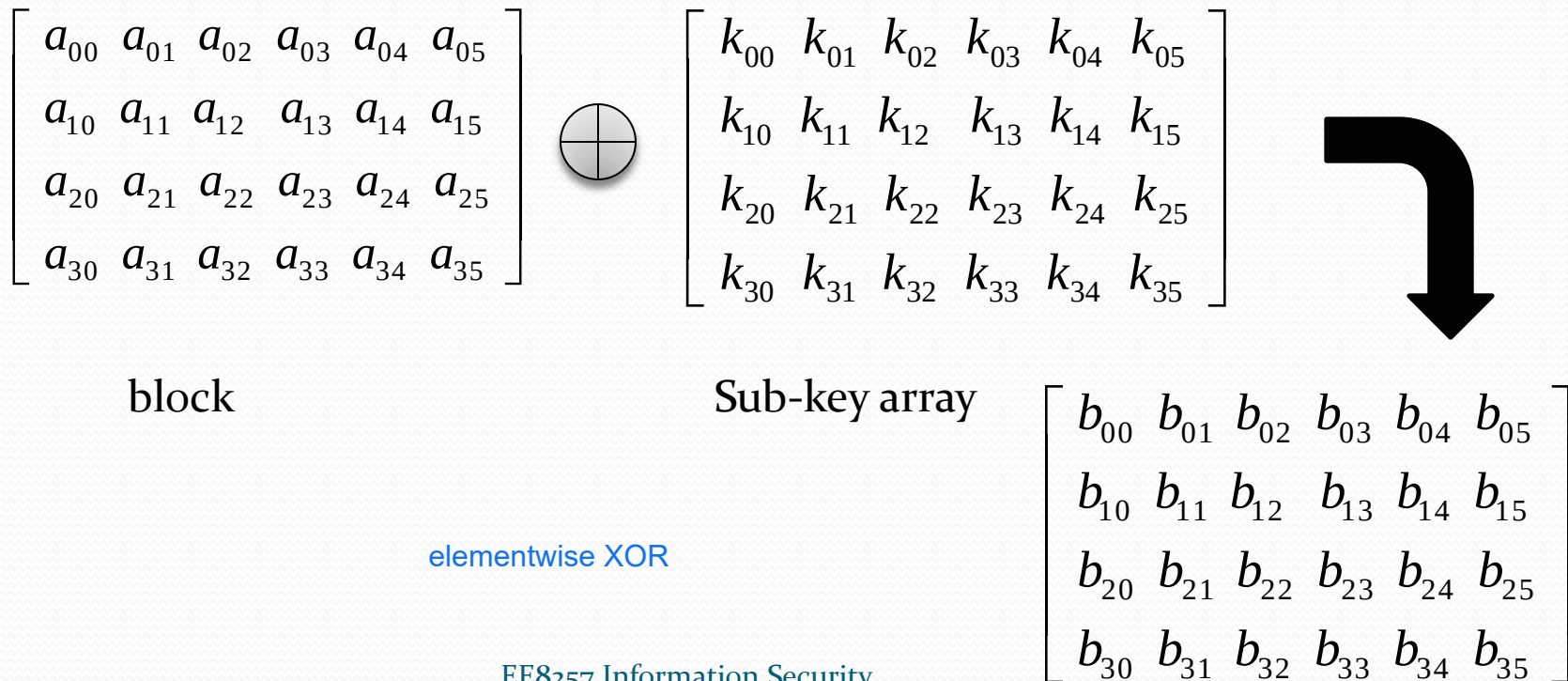
- This operation is applied to column wise
- Includes a shift and a XOR operation
- Column elements are XORed with an elements of a fixed size matrix $\rightarrow$ Lookup table

$$b_{ij} = \text{Shift}(a_{ij} \oplus c_{ij})$$

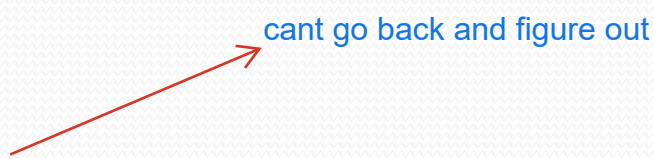


AES Add Round Key

- Key schedule / generation algorithm generates the sub-key byte matrix
- Each element in the arrays are XORed to obtain the cipher text elements → array



AES Decryption

- All the functions / steps of the AES are invertible
- ByteSub is invertible
- ShiftRow is invertible
- MixColumn is invertible
- AddRoundKey is invertible  cant go back and figure out
- Decryption is possible

Tiny Encryption Algorithm (TEA)

skipped

- Not a Feistel cipher
- Encryption scheme is simpler than other block ciphers
- Block length : 64 bits
- Key size : 128 bits
- 32 bit computing architecture $\rightarrow 2^{32}$ modulo
- Number of rounds are variable and higher than other block ciphers. (Round function is simpler) $\rightarrow 32$ considered as secure
- Uses addition and subtraction along with XOR
- Variants of TEA : XTEA and STEA

TEA Encryption

skipped

```
(K[0], K[1], K[2], K[3]) = 128 bit key
(L, R) = plaintext (64 - bit block)
delta = 0x9e3779b9
sum = 0
for i = 1 to 32
    sum = sum + delta
    L = L + (((R << 4) + K[0]) ⊕ (R + sum) ⊕ ((R >> 5) + K[1]))
    R = R + (((L << 4) + K[2]) ⊕ (L + sum) ⊕ ((L >> 5) + K[3]))
next i
ciphertext = (L, R)
```

TEA Decryption

skipped

```
(K[0], K[1], K[2], K[3]) = 128 bit key
(L, R) = ciphertext (64 - bit block)
delta = 0x9e3779b9
sum = delta << 5
for i = 1 to 32
    R = R - (((L << 4) + K[2]) ⊕ (L + sum) ⊕ ((L >> 5) + K[3]))
    L = L + (((R << 4) + K[0]) ⊕ (R + sum) ⊕ ((R >> 5) + K[1]))
    sum = sum - delta
next i
plaintext = (L, R)
```

Overview on other Block Ciphers

- International Data Encryption Algorithm (IDEA)
 - Mixed mode arithmetic
 - combine modulo 2 and modulo 2^{16}
 - non-linear (no S-box)
- Blowfish
 - Key dependent S-box
- RC6

Block Cipher Modes

- Stream cipher generates a key stream with the same size as plaintext
- In block ciphers, encrypting one block is easy
- To encrypt multiple blocks → New key for each block is required
- Solution → Make encryption of a certain block dependent on the previous block (Chaining)

Block Cipher Modes (Cont...)



- Electronic Code Book (ECB) Mode
- Cipher Block Chaining (CBC) Mode
- Counter Mode (CTR) Mode

ECB Mode

- Encrypt each block independently with the same key
- Plaintext blocks $\rightarrow P_0, P_1, P_2, \dots, P_n$
- Cipher text blocks $\rightarrow C_0, C_1, C_2, \dots, C_n$
- Encryption Formula $\rightarrow C_i = E(P_i, K)$ for $i = 0, 1, 2, \dots$

Encryption

$$C_0 = E(P_0, K)$$

$$C_1 = E(P_1, K)$$

$$C_2 = E(P_2, K)$$

Decryption

$$P_0 = D(C_0, K)$$

$$P_1 = D(C_1, K)$$

$$P_2 = D(C_2, K)$$

- This will create a codebook (lookup table) for a fixed key

ECB cut and paste attack

- Plaintext → “Trudy Shoot Alice. Bob Saves Alice.”
- If each word represents a block → $P_0, P_1, P_2, P_3, P_4, P_5$
- Encryption would give → $C_0, C_1, C_2, C_3, C_4, C_5$
- After *cut-and-paste* → $C_0, C_4, C_2, C_3, C_1, C_5$
- Decrypted as → “Trudy Saves Alice. Bob Shoots Alice.”
- This is a critical issue !!! 
- With the codebook, If $P_i = P_j \rightarrow C_i = C_j$ and vice versa
- Same plaintext block produces the same ciphertext block
- Cryptanalysts could exploit this weakness in ECB



Image encryption in ECB

- Alice's image → uncompressed and encrypted with ECB



this violates CIA confidentiality
becz noone shouldnt be able to
access others data



ENCRYPTED IMAGE with
ecb mode

- Obscurity is not accomplished

Image encryption in ECB (Cont...)



Original Image



Encrypted using
ECB mode



Encrypted using
other mode

CBC Mode

P = plain text
K = key
IV = initialization vector
C = cipher text

- Blocks are chained
- Ciphertext from a block is used to obscure the plaintext before encryption
- Encryption Formula $\rightarrow C_i = E(P_i \oplus C_{i-1}, K)$ for $i = 0, 1, 2, \dots$
- Randomly selected **Initialization Vector (IV)** is used to initialize the CBC. IV is not a secret.

Encryption

$$C_0 = E(P_0 \oplus IV, K)$$

$$C_1 = E(P_1 \oplus C_0, K)$$

$$C_2 = E(P_2 \oplus C_1, K)$$

Decryption

$$P_0 = D(C_0, K) \oplus IV$$

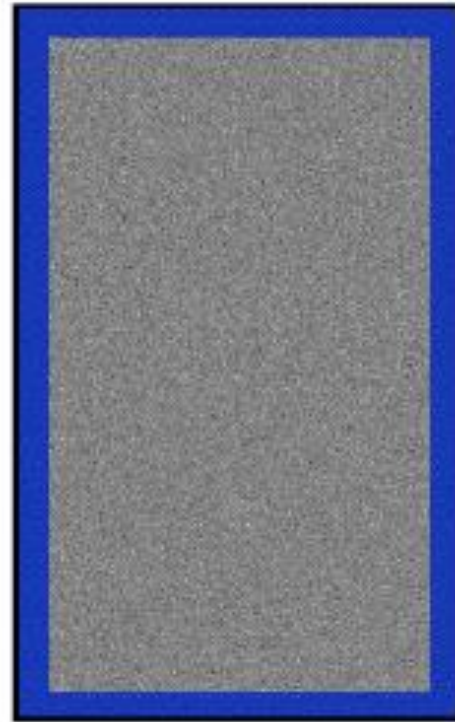
$$P_1 = D(C_1, K) \oplus C_0$$

$$P_2 = D(C_2, K) \oplus C_1$$

CBC Mode (Cont...)

- Identical plaintext blocks produce different ciphertext blocks
- Cut-and-paste is still possible → more complex
- In case of errors (transmission): If one block is received with errors
- If C_i is received as $G \rightarrow P_i \neq P'_i = D(G, K) \oplus C_{i-1}$
 $P_{i+1} \neq P'_{i+1} = D(C_{i+1}, K) \oplus G$
- But, $P_{i+2} \neq P'_{i+2} = D(C_{i+2}, K) \oplus C_{i+1}$
- Therefore, CBC automatically recovers from errors →
Though, a single bit error could sabotage two blocks

CBC Image Encrypting



Counter Mode (CTR)

- Plaintext is encrypted with a key stream (different key for each plaintext block)
- Block cipher is used as a stream cipher
- Encryption formula $\rightarrow C_i = P_i \oplus E(IV + i, K)$ for $i = 0, 1, 2, \dots$

Encryption

$$C_0 = P_0 \oplus E(IV, K)$$

$$C_1 = P_1 \oplus E(IV + 1, K)$$

$$C_2 = P_2 \oplus E(IV + 2, K)$$

Decryption

$$P_0 = C_0 \oplus E(IV, K)$$

$$P_1 = C_1 \oplus E(IV + 1, K)$$

$$P_2 = C_2 \oplus E(IV + 2, K)$$

in counter mode decryption also encryption E is here

- CTR is used for random access. (So does CBC)

Integrity Checking using Block Ciphers

- Integrity - Prevent unauthorized modification of data
- Encrypting a text alone would not assure integrity
 - ECB cut-and-paste attack
- Up to now, we have discussed the use of block ciphers for encryption
- Block ciphers can be used for checking whether a message has been tampered with or not

Message Authentication Code (MAC)

- MAC is generated using the CBC block cipher mode
 - CBC residue
- MAC is used to check the integrity of a message

MAC Generation Process :

- Encrypt the data using CBC mode
- Discard the ciphertext blocks except the last one
- MAC = Final cipher text block

MAC Computation

- For N blocks
$$C_0 = E(P_0 \oplus IV, K)$$
$$C_1 = E(P_1 \oplus C_0, K)$$
$$C_2 = E(P_2 \oplus C_1, K)$$

•

•

this final block is used as MAC bcz
to check whether the msg is altered or not


$$C_{N-1} = E(P_{N-1} \oplus C_{N-2}, K) = \text{MAC}$$

- MAC is attached to the encrypted message and sent along with it
- MAC is again computed at receiving end on the decrypted plaintext blocks and matched with the received one



MAC Integrity Violation

- In case of generating a MAC for 4 blocks

$$\begin{array}{ccccc} C_0 = E(P_0 \oplus IV, K) & \searrow & C_2 = E(P_2 \oplus C_1, K) & \searrow & C_4 = E(P_4 \oplus C_3, K) \\ C_1 = E(P_1 \oplus C_0, K) & \searrow & C_3 = E(P_3 \oplus C_2, K) & \searrow & C_5 = E(P_5 \oplus C_4, K) \end{array}$$

- If the first plaintext block P_1 is changed to X by Trudy

$$\begin{array}{ccccc} C_0 = E(P_0 \oplus IV, K) & \searrow & \tilde{C}_2 = E(P_2 \oplus \tilde{C}_1, K) & \searrow & \tilde{C}_4 = E(P_4 \oplus \tilde{C}_3, K) \\ \tilde{C}_1 = E(X \oplus C_0, K) & \searrow & \tilde{C}_3 = E(P_3 \oplus \tilde{C}_2, K) & \searrow & \tilde{C}_5 = E(P_5 \oplus \tilde{C}_4, K) \end{array}$$

- The change is propagated to the final block
- Therefore, $MAC_{Received} \neq MAC_{Computed}$

MAC Integrity Violation (Cont...)

MAC act as the checksum

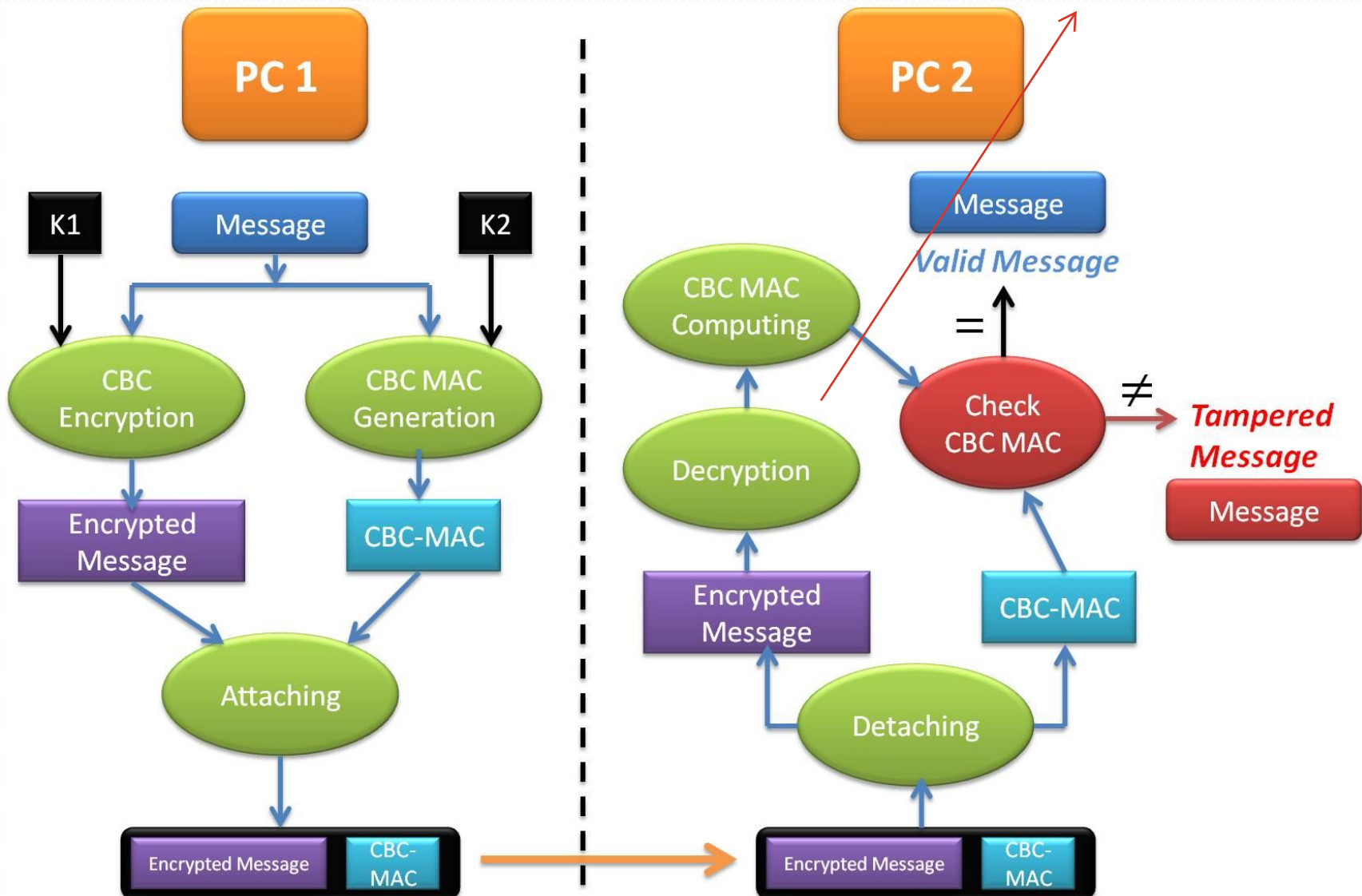
- Trudy needs the key to change the MAC effectively
- Encrypting and MAC generation with the same key would be insecure
 - Sending the final block as the MAC
- Solution would be to;
 - Encrypting with one key
 - MAC generation with another key



But takes more work

MAC Checking Process

we use the MAC and generate the MAC again



Uses of Symmetric Key Crypto

- Confidentiality
 - Transmitting and storing data securely in insecure channel and media
- Integrity Protection
 - Through MAC checking